631 Final Project Report - A Customized User-interactive Movie Query and Recommendation System

William Pan S01435755
Ray Zhang S01435771
Zida Wang S01401660

1 Introduction

In today's world, where we are spoilt for choice when it comes to movies, it can often be overwhelming to decide which film to watch. With so many options available, it can be challenging to find a movie that perfectly matches people's moods and preferences. To address this issue, we built a customized user-interactive movie query and recommendation system.

Our system has three main functionalities: the first one is the login functionality to identify a registered user based on his/her userID. Once a user has successfully logged into our system, our query functionality provides search boxes for genres, comment tags, and ratings that enable the user to search for movies based on his/her preferences. The last functionality of our system is to utilize machine learning algorithms, SVD specifically, to analyze a user's viewing history and generate a personalized recommendation movie list. Our application aims to simplify the decision-making process by providing personalized movie recommendations based on user ratings. By doing so, we hope to save users time and effort and enhance their overall movie-watching experience. The user could make and save a want list based on the filtered recommendation list for later watching references.

We believe our application is important in the sense that people often have limited free time, and movie-watching is a popular activity during holidays and weekends. However, finding the right movie can be time-consuming and frustrating, leading to users settling for subpar films or not watching anything at all. Our application can help users maximize their leisure time by providing them with a curated list of films that align with their preferences, ultimately enhancing their movie-watching experience and enjoyment. Whether the users are in the mood for a comedy, romance, or action movie, our application provides a comprehensive selection of films that are sure to keep them entertained.

2 Prior Work

"Netflix Prize and SVD-Based Collaborative Filtering" by Yehuda Koren et al. This paper describes the winning algorithm from the Netflix Prize, which used SVD-based collaborative filtering to make movie recommendations. The authors applied SVD to the user-item matrix to extract latent factors that capture the underlying characteristics of the movies and user preferences. The resulting model was able to make accurate recommendations and outperformed other collaborative filtering algorithms. We drew on the application of SVD-based collaborative filtering to perform our recommendation functionality by training the default SVD model from Python's Surprise library.

"Scalable Semantic Search with Solr" by Tommaso Teofili et al. This paper describes a system for scalable semantic search using Solr. The authors used Solr's faceting and grouping features to enable efficient query execution and result ranking. They also used a semantic similarity measure to improve the relevance of the search results. The resulting system was able to handle large-scale datasets and achieve high search accuracy. We looked at the details of the implementation syntax with Solr and built the query functionality for our system.

3 Dataset

The MovieLens 25M dataset is a collection of movie ratings provided by users of the MovieLens website. It contains 25000095 ratings and 1093360 tag applications across 62423 movies. The dataset includes a range of information about ratings, movies, the tags for each movie posted by users, and so on. The MovieLens 25M dataset has been used in numerous research papers and competitions related to movie recommendation systems, such as the Netflix Prize competition. It is often used as a benchmark dataset for evaluating the performance of new recommendation algorithms and comparing them to existing methods.

4 Model/Algorithm/Method

SVD

Singular Value Decomposition (SVD) is a matrix factorization technique commonly used in collaborative filtering for recommendation systems. Mathematically, SVD factorizes the sparse matrix where the rows represent the users and the columns represent the movies and

each cell in the matrix represents the rating given by a user to a movie. By performing dimensionality reduction, we can select the top-k singular values and the corresponding latent feature vectors for a user/movie, which will remove noise from the data and make the computation faster. We can then take the dot product of the embedded latent vectors to predict the rating for the given user-movie pair. In our application, instead of manually performing SVD, we utilized the built-in SVD model in Python's Surprise library to enhance the recommendation process. We recommend a personalized list to a user given the input of userID based on the top predicted ratings from his/her unwatched movies.

Data Crawling

In task 1, we efficiently scraped the top 20 reviews for each movie on the IMDB website using the Requests and BeautifulSoup modules. Requests is utilized for sending HTTP requests to websites, while BeautifulSoup handles parsing the responses from these requests. In the beginning, we naively looped through the IMDB movieID list, which took us 2-3 hours for 10k websites. Then, we optimize it by implementing multiprocessing, and it significantly sped up the process. When we set to 10 processes to deal with the requests simultaneously, the time spent on requesting and parsing 10k websites was reduced to 20 mins, which is a huge speed-up compared to the old single-threaded design.

Connection to Apache Solr

We employ the pysolr package in Python to interface with Apache Solr. First, we use the pysolr.Solr() method to obtain an instance of a core in Apache Solr. Next, we transform CSV file rows into a list of documents and utilize the solr.add() method to upload our data to the Apache Solr core. To fetch data from Solr, the solr.search() method is used. By leveraging our knowledge of Apache Solr query commands, we can customize the parameters in solr.search() to collect data according to specific requirements.

Retrieve Data

We have two different kinds of retrieved data. One is movie recommendations for a specific user, the other is movie searching based on keywords. Recommendations leverage the user's history rating for different movies. We use SVD to compare the characteristics of the movies he has rated before to the movies that he has never seen. In this way, we give prediction ratings for the movies he has never seen and show the movies with high scores. On the other hand, we can also retrieve data based on specific features, which are movie names, movie

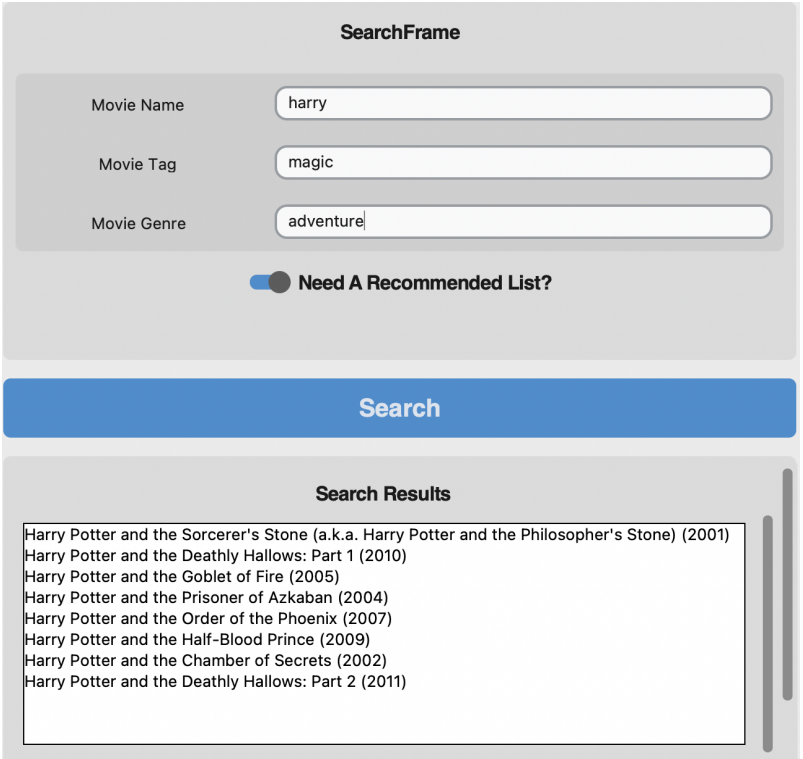
tags, and movie genres. After typing in some of these features, we can give the result that contains the requested features.

Movie List

We developed a user-friendly movie-list box in tkinter that allows users to save their favorite movies for holiday viewing. Users can simply click on a movie name in the search or recommendation box and then click the save button to add the selected movie to their movie-list box. Ultimately, users have the option to export their movie-list box to a CSV file for easy access.

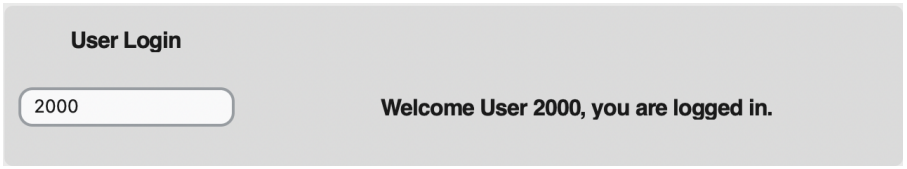
5 Results and findings

Search Functionality



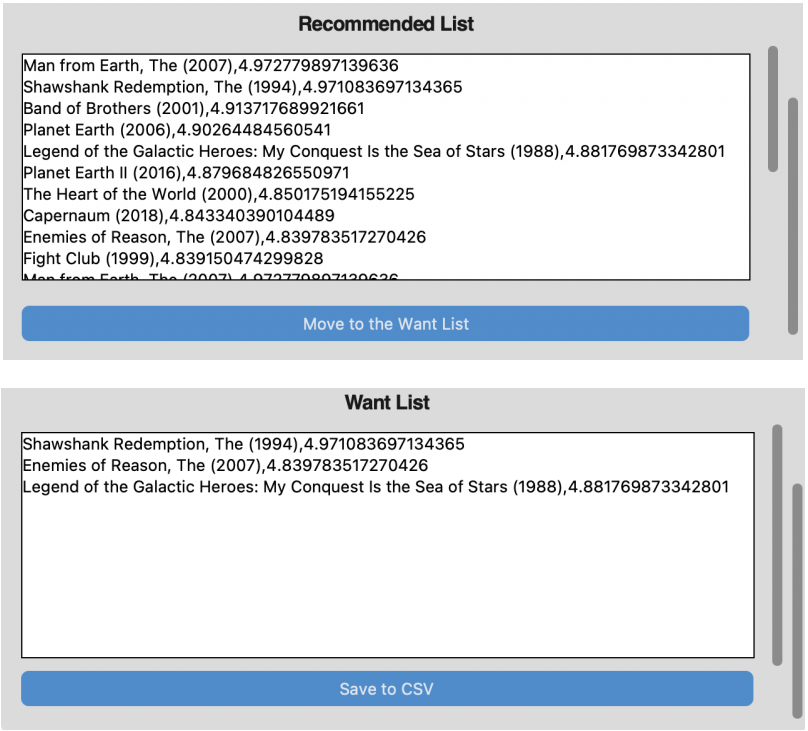
Our advanced search functionality provides users with the ability to input customized query terms in the search box to generate tailored results. When multiple fields receive input, we employ Solr's "AND" operation to ensure all specified query criteria are addressed. To avoid overloading the results display, we limit the number of retrieved records to a maximum of 20.

User-interactive Login



Our user-interactive login functionality greets users with a welcome message upon entering their userID. Concurrently, if users enable the option for receiving a recommendation list, we generate a personalized selection of items ranked by predicted ratings. This tailored list is created using a pre-trained SVD (Singular Value Decomposition) model, ensuring a customized experience for each user.

Generating and Downloading Want List



We have also effectively implemented a user-interactive feature that enables the creation and saving of a personalized want list from both search results and the model-generated recommendation list. Users can curate their want list by selecting items from these sources, and the list can be conveniently exported as a CSV file for future reference.

While our application has been successful overall, there are still some areas for improvement to enhance the user experience. Firstly, our application may not immediately detect an incorrect userID, which could adversely impact subsequent processes. Although users can erase the userID input field, the welcome log retains the first digit of the previously entered userID even when the field is empty. This unusual behavior remains unresolved. Secondly, the recommendation list displays movie titles followed by predicted ratings with multiple decimal places. The want list, which is created by selecting records from both the search results and the personalized recommendation list, may exhibit aesthetic inconsistencies in its visualization due to the different formats of the two data sets. Addressing these issues will help to create a smoother, more visually appealing user experience.

References

1 Teofili, T., Ciceri, E., Mambrini, F., & Stellato, A. (2014). Scalable semantic search with 2 Solr. Journal of Web Semantics, 24, 22-28. https://doi.org/10.1016/j.websem.2013.12.001

2 Koren, Y., Bell, R., & Volinsky, C. (2009). The Netflix prize and SVD-based collaborative filtering. IEEE Internet Computing, 13(5), 43-49. https://doi.org/10.1109/MIC.2009.109

3 Harper, F. M., & Konstan, J. A. (2015). The MovieLens datasets: History and context. ACM Transactions on Interactive Intelligent Systems (TiiS), 5(4), 19:1-19:19. https://doi.org/10.1145/2827872